Foreman graph schema, v1.1

What the graph stores, how it is linked, and how well the conflict check works on the real TinyDB code.

Status: the design was approved on Oct 15, and the real TinyDB code is now loaded and measured (Section 9). Written in plain words. Everything here was tested on a local copy of the same database engine. Your Docker runs are the final proof.

1. The idea in one minute

- There is **one graph per repo**. A repo is a tenant. Nothing in one graph can be seen from another.
- Inside each graph there are **three layers**:
  - **Code layer:** the map of TinyDB (files, functions, tests, and who calls what).
  - **Work layer:** what is happening now (agents, tickets, claims, messages, handoffs, results, decisions).
  - **Memory layer:** what the crew has learned (what happened, what is true, how to fix things).
- Tickets and memories both point at code. That is how a worker finds old lessons about the code it is about to change.
- Every choice the crew makes is saved as a **decision** you can click on and explain.

2. The five Track 2 focus areas and where each one lives

Focus area	Where it lives in the graph	How we show it works
Episodic, semantic and procedural memory	`Memory` things, each tagged `episodic`, `semantic` or `procedural`, linked to the code they are about	Recall shows all three kinds for one ticket. Success counts raise or lower a recipe's score. Old facts are retired, not deleted.
Persistent conversation history	`Session` and `Message` things. Messages are numbered 1, 2, 3 inside a session	30 messages written at the same instant come back as exactly 1 to 30. A restarted worker reads its own history.
Shared state and handoffs	The ticket's status in the graph, plus `Handoff` notes stored on the ticket	A stale claim reopens the ticket and the next worker finds the previous worker's note.
High-concurrency reads and writes	Every claim, message and memory update is a single database step	50 workers claim at once and exactly one wins. Full load test comes later.
Separate memory per user or tenant	One graph per repo, named `foreman_<repo>`	A secret saved for repo A is invisible from repo B.

3. The things in the graph

Code layer (loaded once from the repo)

Thing	What it stores
`File`	path
`Function`	full name (for example `tinydb.table.Table.update`), short name, file, first and last line, `test_count` (how many tests run it)
`Test`	full test name and whether it passed or failed on the seeded code

Work layer (changes all the time)

Thing	What it stores
`Agent`	id, role (foreman, worker, reviewer or historian), status

Thing	What it stores
`Task`	run id, id, title, description, status (open, claimed, submitted, approved), who holds it, when the claim runs out, attempt count
`Session`	one conversation between an agent and a ticket; remembers its last message number
`Message`	session id, number, who spoke, the text, the time
`Handoff`	done, remaining, tried, files touched, time
`Result`	summary, tests passed, files touched, patch reference
`Decision`	kind (claim, footprint, order, review), summary, time

Memory layer (what the crew learned)

Thing	What it stores
`Memory`	kind (episodic, semantic or procedural), summary, status (active or retired), times used, times it worked

That is 11 kinds of things. It is more than the 8 I first said, because conversation history and results each need their own.

4. The links

Link	From to	Meaning
`DEFINED_IN`	Function to File	where the function lives
`CALLS`	Function to Function	who called whom while the tests ran, with a count. Observed by running the tests, because TinyDB forwards many calls in ways that reading the text cannot follow
`IMPORTS`	File to File	which file imports which
`COVERS`	Test to Function	this test ran this function, with a count
`FAILS_ON`	Task to Test	the ticket's failing tests
`TOUCHES`	Task to Function	the ticket's footprint. Each link carries a weight: how suspicious the function is for this ticket, times how specific it is
`CONFLICTS_WITH`	Task to Task	the two tickets share code, with a score
`HAS_SESSION`, `HAS_MESSAGE`	Agent to Session, Session to Message	conversation history
`FOR`	Session, Handoff and Result to Task	which ticket it belongs to
`WROTE`, `SUBMITTED_BY`	Agent to Handoff, Result to Agent	who wrote it
`MADE_BY`, `ABOUT`	Decision to Agent, Decision to Task	who decided what about which ticket
`ABOUT`	Memory to Function	what the memory is about
`LEARNED_FROM`	Memory to Task	where the memory came from
`SUPERSEDED_BY`	Memory to Memory	this fact was replaced by that one

5. How the rules are kept

- **Safe claim:** one database step checks the ticket is open, checks no conflicting ticket is already claimed, claims it, and records the decision. Nothing can slip in between.
- **No duplicates:** the database refuses a second `Task` with the same run and id, a second `Agent` with the same id, or two messages with the same number in one session.
- **Conflicts count specific code more.** For every function the ticket's failing tests ran, the weight is *suspicious* times *specific*. Suspicious means the function shows up in this ticket's failing tests but not in passing ones. Specific means few tests run it, so a function every test runs counts for nothing. Two tickets' overlap is the shared weight divided by their combined weight, from 0 to 1. At 0.10 or more they are marked as conflicting.

- **Memory updates over time:** each memory keeps `uses` and `successes`. Its score is `(successes + 1) / (uses + 2)`, so new memories start at a fair middle score.
- **Runs do not mix:** work-layer things carry a `run_id`. Memory is cleared at the start of each full evaluation run, except in the restart demo.

6. What was tested

All of this ran on my side, on the same engine family (FalkorDB 4.18), six times in a row with no failures:

- Setup creates 9 uniqueness rules and can be run twice safely.
- The same conflict query is used on the toy graph and on the real TinyDB graph. On the toy graph it separates specific shared code (flagged) from busy shared code (not flagged).
- 50 workers, split across two conflicting tickets, claimed at the same instant: exactly 1 winner, exactly 1 claim decision recorded, and the other ticket blocked.
- Explain returned who decided, which ticket, which code, and which tickets were blocked.
- 30 messages written at once came back numbered 1 to 30, none lost, none repeated.
- A stale claim reopened and the handoff note was waiting for the next worker.
- All three memory kinds were recalled for a ticket, a retired fact stopped appearing, and a recipe's score went up after two successes.
- A secret in repo A was invisible from repo B.
- A duplicate agent was refused by the database.

Not yet tested: these scripts on your Docker image. The numbers in the small example graph are made up. The real-code numbers are in Section 9.

7. Changes this causes to the locked contract

The contract has no way to save conversation history. Adding it means **contract v1.1**, with four new functions:

New function	What it does
`start_session(task_id, agent_id)`	opens a conversation for a ticket
`log_message(session_id, role, content)`	saves one message, numbered safely
`get_history(session_id)`	returns the messages in order
`rate_memory(memory_id, worked)`	records whether a recalled memory helped

Everything else in v1 stays the same. Your teammate has not seen v1 either, so the whole thing goes through the change note when they are back.

8. Second contract change: v1.2 (approved Oct 15)

How footprints are built changed after measuring (Section 9). That touches two return shapes in the locked contract:

- `get_footprint(task_id)` returns `{files, functions: [{id, weight}], tests, decision_id}`. The old `max_hops` goes away, because the footprint no longer walks out by calls. The test trace already includes everything the failing tests reached through calls.
- `find_conflicts(task_id)` returns `{other_task_id, other_status, score, shared_nodes}`. The old `hops` becomes `score`.
- Locked decision 2 now reads: "Footprints are the functions the ticket's failing tests ran, weighted by suspiciousness and specificity."

Nothing else in the contract changes. This is applied in contract v1.2.

9. Measured on the real TinyDB code

The graph now holds the seeded TinyDB: 10 files, 117 functions, 228 tests (35 failing, 193 passing), 173 observed call links and 3,658 test-to-function links. The 10 tickets were scored against each other, and then compared with the hidden answer key (evaluation only).

- **The real fix function was inside the footprint for 10 of 10 tickets.** Workers are pointed at the right area.
- **Ticket pairs with any shared code: 19 of 45.** The other 26 pairs share nothing, so they can always run in parallel.
- **At the 0.10 cut-off, 6 pairs are marked as conflicting.**
- **The "update" pair (the key's second pair) ranked 1st with score 0.24.** The ticket about `add()` and the ticket about `update()` share `Table.update` and the code around it.
- **The "ids restart" pair ranked 3rd with score 0.15.** Found, but they share mostly busy code, so the score is modest.
- **The "cache" pair was not found (rank 10, score 0.08).** This is the known mismatch: one of its bugs sits in `LRUCache.__getitem__`, which the other bug's code never calls. No code-based check can link them. This stays as a documented limit.
- **4 extra pairs were flagged that the key did not intend.** They are all between tickets that really do run the same table and storage code, so they are plausible, but they cost some parallelism.

A problem found and fixed on the way: when a test fails, pytest prints the objects involved, which calls their `__repr__` functions. Those calls only happen in failing tests, so they looked like clues about the bug and pushed unrelated tickets together. The tracer now ignores any call that pytest itself started. After that fix, the real fix function was the most suspicious function for 7 of 10 tickets and within the top 3 for all 10.

What is still open

- **Cut-off:** 0.10 is a good middle, but it is a choice. A lower one flags more pairs and finds the "ids" pair more reliably, a higher one allows more parallel work. We can test both in the three-setup evaluation.
- **Claim timeout and heartbeat numbers:** still deferred until we measure how slow the models are.
- **Reviews:** the explain query handles claims. Review decisions point at results, so it gets one more case when we write the full set of functions.
- **Results:** the `Result` thing is defined, but its queries come with the full set of functions.

10. The functions are built (`graph_api.py`)

All 26 functions in contract v1.2 now exist as one Python class (plus two housekeeping calls for the evaluation, `clear_memory` and `reset_run`, which agents never see), `GraphAPI(repo, run_id)`. Each one runs Cypher in FalkorDB. 14 contract tests cover them, including 50 workers claiming at once, 30 messages written at once, expiry and handoff, the review cycle, memory ranking, tenant separation and run separation. All 14 passed five times in a row on my side, and the real TinyDB check inside them reproduces the 6 conflict pairs.

Small choices I made where the contract was silent (none changes a name or shape in the contract):

- **Lists come under a named key**, because every function returns a dict with `ok`: `tasks`, `conflicts`, `memories`, `messages`.
- **`get_handoff` returns `note`** (the note, or `None`).
- **`start_session` is safe to call twice.** A restarted agent gets its old session back (`resumed: true`) and carries on from its history.
- **IDs are `dec-<number>`, `mem-<number>`, `res-<number>`, `sess-<number>`.** The number comes from a counter in the graph, so ids never repeat. A task's id is its ticket id (for example `T-01`).
- **`get_result` also returns `out_of_footprint`:** the files the worker touched that are not in the ticket's footprint. That is what the Reviewer checks.
- **`recall_similar` ranks** by how targeted a memory is (the share of its functions the ticket touches) times how reliable it is (`(successes + 1) / (uses + 2)`). A recipe that keeps failing sinks below one that works.

- **Claim, review, submit, release and expiry each do their check, change and decision record in one database step**, so there is no gap for two workers to slip through.
- **Only `claimed` tickets block a conflicting claim**, as the contract says. A ticket waiting for review does not.

What the run metrics mean (`get_run_metrics`):

- **`claims_attempted`:** granted plus refused, with the refusals split by reason.
- **`double_claims`:** a ticket held twice with no release, expiry or rejection in between. It should always be 0. It is a check that would catch a bug.
- **`collisions`:** two tickets whose results touched the same file while both were being worked on.
- **`reworks`:** how many times reviews sent a ticket back.
- **`handoffs_recovered`:** tickets that had a handoff note and were claimed again afterwards.

Not built yet: the load test, the in-memory stub for agent development, and the tool server that wraps these functions for the agents.